Major Assignment

1. Derivation of the Kernel $\widetilde{K}$

We are given the non–parametric kernel acting on $z \in \mathbb{R}^2$:

$$K(z_1, z_2) = (z_1^\top z_2 + c)^d,$$

with feature map $\phi : \mathbb{R}^2 \to \mathcal{H}$ satisfying

$$K(z_1, z_2) = \phi(z_1)^\top \phi(z_2).$$

The semi–parametric model is

$$y = w(z)x + b, \qquad w(z) = p^\top \phi(z),$$

so

$$y = p^\top \phi(z)\, x + b.$$

We convert this to a purely non–parametric model by defining a new feature map

$$\psi : \mathbb{R} \times \mathbb{R}^2 \to \widetilde{\mathcal{H}}, \qquad \psi(x, z) = \begin{bmatrix} x\,\phi(z) \\ 1 \end{bmatrix}, \qquad \widetilde{p} = \begin{bmatrix} p \\ b \end{bmatrix}.$$

Using the inner product in $\widetilde{\mathcal{H}} = \mathcal{H} \oplus \mathbb{R}$,

$$\left\langle \begin{bmatrix} a \\ \alpha \end{bmatrix}, \begin{bmatrix} b \\ \beta \end{bmatrix} \right\rangle = a^\top b + \alpha\beta,$$

we compute

$$\widetilde{p}^\top \psi(x, z) = \begin{bmatrix} p \\ b \end{bmatrix}^\top \begin{bmatrix} x\phi(z) \\ 1 \end{bmatrix} = p^\top (x\phi(z)) + b \cdot 1 = x\, p^\top \phi(z) + b.$$

Thus the semi–parametric model becomes linear in the feature space $\widetilde{\mathcal{H}}$.

By the kernel trick, the kernel acting on pairs (x, z) is

$$\widetilde{K}\big((x_1, z_1), (x_2, z_2)\big) = \psi(x_1, z_1)^\top \psi(x_2, z_2).$$

Compute:

$$\psi(x_1, z_1)^\top \psi(x_2, z_2) = \begin{bmatrix} x_1\phi(z_1) \\ 1 \end{bmatrix}^\top \begin{bmatrix} x_2\phi(z_2) \\ 1 \end{bmatrix} = x_1 x_2\, \phi(z_1)^\top \phi(z_2) + 1.$$

Using the given polynomial kernel identity

$$\phi(z_1)^\top \phi(z_2) = (z_1^\top z_2 + c)^d,$$

we arrive at

$$\boxed{\widetilde{K}\big((x_1, z_1), (x_2, z_2)\big) = x_1 x_2\, (z_1^\top z_2 + c)^d + 1.}$$

2. Selection of Hyperparameters d and c

A grid search was performed over

$$d \in \{1, 2, 3, 4, 5\}, \qquad c \in \{0, 0.1, 1, 2, 5\}.$$

For each pair (d, c):

- the kernel $\widetilde{K}$ was computed,
- a Kernel Ridge Regressor (KRR) with $\alpha = 1$ was trained,
- performance was evaluated on the validation set using the coefficient of determination R^2.

```
Starting Grid Search...
------------------------------------------------------------
Degree (d)   | Coef0 (c)    | Test R2 Score
------------------------------------------------------------
1            | 0            | 0.81005
1            | 0.1          | 0.97024
1            | 1            | 0.97023
1            | 2            | 0.97023
1            | 5            | 0.97023
2            | 0            | 0.64507
2            | 0.1          | 0.97032
2            | 1            | 0.97020
2            | 2            | 0.97020
2            | 5            | 0.97020
3            | 0            | 0.53464
3            | 0.1          | 0.96906
3            | 1            | 0.97018
3            | 2            | 0.97018
3            | 5            | 0.97018
4            | 0            | 0.46242
4            | 0.1          | 0.95123
4            | 1            | 0.97016
4            | 2            | 0.97017
4            | 5            | 0.97018
5            | 0            | 0.40364
5            | 0.1          | 0.88781
5            | 1            | 0.97023
5            | 2            | 0.97023
5            | 5            | 0.97023
------------------------------------------------------------
OPTIMAL PARAMETERS FOUND:
  Degree (d)    = 2
  Coefficient (c) = 0.1
  Best R2 Score   = 0.97032
```

Figure 1: R^2 scores for various C and d values in polynomial Kernel Ridge Regression.

The best validation performance occurred at:

$$\boxed{d^* = 2, \qquad c^* = 0.1, \qquad R^2_{\text{best}} = 0.97032.}$$

These values are therefore used in the final submitted kernel implementation.

3. Code for the Custom Kernel $\widetilde{K}$

The kernel derived in Question 1,

$$\widetilde{K}\big((x_1, z_1), (x_2, z_2)\big) = x_1 x_2 (z_1^\top z_2 + c)^d + 1,$$

was implemented in Python using only NumPy and `sklearn.metrics.pairwise.polynomial_kernel` for the polynomial part.

Given inputs

$$X_1 \in \mathbb{R}^{n_1 \times 1},\ Z_1 \in \mathbb{R}^{n_1 \times 2},\ X_2 \in \mathbb{R}^{n_2 \times 1},\ Z_2 \in \mathbb{R}^{n_2 \times 2},$$

the kernel matrix is computed as

$$\widetilde{K}_{ij} = X_{1,i} X_{2,j}\,(Z_{1,i}^\top Z_{2,j} + c)^d + 1.$$

This implementation is used in the final `submit.py` file.

4. Inverting a 1089-Dimensional XOR Arbiter PUF Model

A k-XOR Arbiter PUF composed of two 32-stage Arbiter PUFs produces a linear model of dimension $33 \times 33 = 1089$. Each underlying Arbiter PUF is characterized by 32 delay parameters in each of the four internal paths, giving

$$32 \times 4 \times 2 = 256 \text{ non-negative delays.}$$

The goal is to outline how these 256 delays generate the 1089-dimensional linear model, and how the process can be inverted.

4.1 Generating a Linear Model from Delays

Each 32-stage Arbiter PUF consists of four delay vectors:

$$p, q, r, s \in \mathbb{R}^{32}_{\geq 0}.$$

These correspond to the four possible internal signal transitions at a stage. Define the per-stage differential delays

$$\alpha_i = p_i - q_i, \qquad \beta_i = r_i - s_i.$$

For a single Arbiter PUF, the classical additive delay model gives the weight vector $w \in \mathbb{R}^{33}$:

$$w_0 = \alpha_0, \qquad w_i = \alpha_i + \beta_{i-1}\ (1 \leq i \leq 31), \qquad w_{32} = \beta_{31}.$$

In matrix form this is a linear map

$$w = A \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \qquad A \in \mathbb{R}^{33 \times 64}.$$

For a 2-XOR Arbiter PUF, the final model becomes the outer product:

$$W = w^{(1)} \left(w^{(2)}\right)^\top,$$

where $W \in \mathbb{R}^{33 \times 33}$ is reshaped into a 1089-dimensional vector.

4.2 Inverting the Linear Model

Because W is the outer product of two unknown vectors $w^{(1)}$ and $w^{(2)}$, it is rank-1. Thus we may recover them using Singular Value Decomposition (SVD):

$$W = U\Sigma V^\top$$

and extract

$$w^{(1)} = \sqrt{\sigma_1}\, U_{:,1}, \qquad w^{(2)} = \sqrt{\sigma_1}\, V_{1,:}^\top.$$

Each vector w must be inverted back to (α, β). The forward model gives the equations:

$$\alpha_0 = w_0, \qquad \alpha_i + \beta_{i-1} = w_i, \quad 1 \le i \le 31, \qquad \beta_{31} = w_{32}.$$

This system is underdetermined. One simple inversion approach is:

$$\beta_0 = \beta_1 = \cdots = \beta_{30} = 0,$$

which yields

$$\alpha_i = w_i \quad (0 \le i \le 31), \qquad \beta_{31} = w_{32}.$$

4.3 Recovering Non-Negative Delays

Using the original definitions:

$$p_i - q_i = \alpha_i + \beta_i, \qquad r_i - s_i = \alpha_i - \beta_i,$$

we can recover non-negative delays via rectification:

$$p_i = \max(\alpha_i + \beta_i,\, 0), \qquad q_i = \max(-(\alpha_i + \beta_i),\, 0),$$

$$r_i = \max(\alpha_i - \beta_i,\, 0), \qquad s_i = \max(-(\alpha_i - \beta_i),\, 0).$$

This guarantees:

$$p_i, q_i, r_i, s_i \ge 0, \qquad p_i - q_i = \alpha_i + \beta_i, \qquad r_i - s_i = \alpha_i - \beta_i.$$

Thus the full set of 256 delays is recovered.